PROGRAM

```
Import heapq


# Heuristic: Manhattan distance
Def manhattan_distance(state, goal):
    Distance = 0
    For val in range(1, 9):  # tiles 1–8, skip 0 (blank)
        Idx1 = state.index(val)
        Idx2 = goal.index(val)
        X1, y1 = idx1 % 3, idx1 // 3
        X2, y2 = idx2 % 3, idx2 // 3
        Distance += abs(x1 – x2) + abs(y1 – y2)
    Return distance



# Get neighbors by moving the blank tile (0)
Def get_neighbors(state):
    Neighbors = []
    Idx = state.index(0)
    X, y = idx % 3, idx // 3
    Directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]  # left, right, up, down

    For dx, dy in directions:
        Nx, ny = x + dx, y + dy
        If 0 <= nx < 3 and 0 <= ny < 3:
```

```
            New_idx = ny * 3 + nx
            New_state = state[:]  # copy state
            New_state[idx], new_state[new_idx] = new_state[new_idx], new_state[idx]
            Neighbors.append(new_state)
    Return neighbors


# Reconstruct solution path
Def reconstruct_path(came_from, current):
    Path = [current]
    While tuple(current) in came_from:
        Current = came_from[tuple(current)]
        Path.insert(0, current)
    Return path


# A* Search
Def a_star(start, goal):
    Open_set = []
    Heapq.heappush(open_set, (manhattan_distance(start, goal), 0, start))
    Came_from = {}
    G_score = {tuple(start): 0}

    While open_set:
        _, cost, current = heapq.heappop(open_set)
```

```
        If current == goal:
            Return reconstruct_path(came_from, current)

        For neighbor in get_neighbors(current):
            Tentative_g = g_score[tuple(current)] + 1
            If tuple(neighbor) not in g_score or tentative_g < g_score[tuple(neighbor)]:
                G_score[tuple(neighbor)] = tentative_g
                F = tentative_g + manhattan_distance(neighbor, goal)
                Heapq.heappush(open_set, (f, tentative_g, neighbor))
                Came_from[tuple(neighbor)] = current

    Return None


# IDA* Search
Def ida_star(start, goal):
    Def dfs(path, g, threshold):
        Node = path[-1]
        F = g + manhattan_distance(node, goal)
        If f > threshold:
            Return f
        If node == goal:
            Return path
        Min_threshold = float('inf')
```

```
        For neighbor in get_neighbors(node):
            If neighbor not in path:  # avoid cycles
                Path.append(neighbor)
                Result = dfs(path, g + 1, threshold)
                If isinstance(result, list):  # found solution
                    Return result
                If result < min_threshold:
                    Min_threshold = result
                Path.pop()
        Return min_threshold

    Threshold = manhattan_distance(start, goal)
    Path = [start]

    While True:
        Result = dfs(path, 0, threshold)
        If isinstance(result, list):
            Return result
        If result == float('inf'):
            Return None
        Threshold = result


# Input
Def get_input():
```

```
    Print("Enter the initial state (9 numbers, use 0 for blank):")
    Start = list(map(int, input().split()))
    Print("Enter the goal state (9 numbers):")
    Goal = list(map(int, input().split()))
    Return start, goal


# Output
Def print_solution(solution):
    If solution is None:
        Print("No solution found.")
    Else:
        Print("Steps to reach goal:")
        For state in solution:
            For I in range(0, 9, 3):
                Print(state[i:i+3])
            Print("-----")
        Print("Total steps:", len(solution) – 1)


# Main
If __name__ == "__main__":
    Start, goal = get_input()

    Print("\n--- A* Search ---")
```

```
A_star_result = a_star(start, goal)

Print_solution(a_star_result)


Print("\n--- Memory-Bounded A* (IDA*) ---")

Ida_star_result = ida_star(start, goal)

Print_solution(ida_star_result)
```